



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

MINI PROJECT

STUDENT COURSE REGISTRATION SYSTEM

Course : SECJ2154 OBJECT ORIENTED PROGRAMMING

Semester/Session : 2-2024/2025

Section : 02

Lecturer : Dr Muhammad Khatibsyarbini

No.	Group Members	Matric No.
1.	Muhammad Adam bin Razali	A23CS0116
2.	Afif Shaqir Irfan bin Arqam	A23CS0204
3.	Muhammad Afiq Danish bin Mohd Hazni	A23CS0118

1.0 INTRODUCTION

The Student Registration System is a desktop application developed in Java, designed to manage the enrollment process for students and course management for administrators. The application utilizes graphical dialog boxes provided by JOptionPane for user interaction, making it easy to navigate without a traditional GUI. The system supports two types of users: students and administrators. Upon successful login, students are allowed to browse through available courses, enroll in those with remaining capacity, drop courses they no longer wish to attend, and review their current course schedule. Administrators, on the other hand, are given the capability to create new courses by entering necessary details such as course name, code, section, credit hours, and seat capacity. They can also remove existing courses, inspect details of any course including who is enrolled, and monitor all registered students. All changes made by either user type are synchronized through a centralized course list, ensuring that seat availability and enrollment records remain consistent across the system.

2.0 ROLES AND SYSTEM WORKFLOW

The workflow of the system differs depending on whether the user is a student or an administrator. Students begin by logging in using their unique student ID and password. Once authenticated, they are presented with options that allow them to view all available courses in the system, enroll in any course that has open seats, drop any course they are currently enrolled in, and check the full list of their current registrations. These functionalities ensure that students have full control over their academic schedules while maintaining data integrity and consistency. Administrators also log in with their assigned credentials. After successful authentication, they are provided with administrative controls that allow them to create new courses by entering course-specific information such as title, section, credit hours, and maximum seat capacity. In addition to creating courses, administrators can remove existing ones, which in turn automatically clears those courses from all student schedules. They can also view a complete list of courses, display detailed information for any selected course including enrolled student IDs, and review a full list of all registered students within the system.

2.0 CLASS IMPLEMENTATIONS

USER

At the core of the system is the User class, which serves as the foundation for all users interacting with the application. Defined in User.java this abstract class contains :

- Login and Logout Functionalities

```
public static User login(String userId, String password, ArrayList<User> allUser) throws
LoginFailedException{ // Exception Handling - Login failed

    for (User user : allUser) {
        if (user.getUserId().equalsIgnoreCase(userId) && user.getPassword().equals(password)) {
            JOptionPane.showMessageDialog(null, "Login successful! Welcome, " + user.getUserId());
            return user;
        }
    }
    throw new LoginFailedException("\nInvalid username or password. Please try again.");
}

Windsurf: Refactor | Explain | Generate Javadoc | X
public void logout(){
    JOptionPane.showMessageDialog(null, "Logout successfully!");
}
```

Figure 2.1: User Class Login and Logout Interface

This method above served as a authentication for the user to enter the system as an administrator or as a Student

- Display Menu Function

```
// Abstract method for polymorphism
Windsurf: Refactor | Explain | X
public abstract void displayMenu();
```

Figure 2.2: Display Menu Interface

This method for displaying menus, which is later implemented differently by its subclasses including The Admin class and Student class.

STUDENT

The Student class extends the User class and is implemented in Student.java. It represents a typical student user and provides the ability to browse all available courses, enroll in courses with remaining seats, drop courses they are currently enrolled in, and view their complete course schedule as visualized below :

- Student Course Registration

```
public void registerCourse() throws DuplicateCourseException { // Exception handling - Detect duplicate course registration
    String courseCode = JOptionPane.showInputDialog(null, "Enter course code:");
    if (courseCode == null) return;

    // Check if courseCode is already registered
    for(Course course : enrolledCourses){
        if(course.getCourseCode().equalsIgnoreCase(courseCode)){
            throw new DuplicateCourseException("You have already registered for this course!");
        }
    }

    for(Course course : courses){
        if(course.getCourseCode().equalsIgnoreCase(courseCode)){
            if(course.getCurrentCapacity() < course.getMaxCapacity()){
                enrolledCourses.add(course);
                course.addCurrentCapacity();
                course.addEnrolledStudent(this.getUserId());
                JOptionPane.showMessageDialog(null, "You have registered for " + course.getCourseName() + " successfully!");
            } else {
                JOptionPane.showMessageDialog(null, "Course is full! Cannot register.", "Error", JOptionPane.ERROR_MESSAGE);
            }
            return;
        }
    }

    JOptionPane.showMessageDialog(null, "Course with code " + courseCode + " not found.", "Error", JOptionPane.ERROR_MESSAGE);
}
```

Figure 2.3: Student Course Registration Process

When a student registers for course, the system will check the input of the Student whether it matches any of the course code present in Course list, if not then it will invoke an error, same if the Student try to enroll for the same Course, the system will throw DuplicateCourseException. the system automatically adjusts the seat count for the respective course, maintaining accuracy in enrollment data.

- Dropping Courses

```
public void dropCourse() {
    String courseCode = JOptionPane.showInputDialog(null, "Enter course code to drop:");
    if (courseCode == null) return;

    for(Course course : new ArrayList<>(enrolledCourses)){
        if(course.getCourseCode().equalsIgnoreCase(courseCode)){
            enrolledCourses.remove(course);
            course.minusCurrentCapacity();
            JOptionPane.showMessageDialog(null, course.getCourseName() + " dropped successfully!");
            return;
        }
    }
    JOptionPane.showMessageDialog(null, "Course not found in your registered courses!", "Error",
    JOptionPane.ERROR_MESSAGE);
}
```

Figure 2.4: Student Course Dropping Process

Same if the user wants to drop course as the image above, the system will check beforehand of the availability of said Course code. Then, the system automatically adjusts the seat count for the respective course, maintaining accuracy in enrollment data.

- Viewing Registered Courses and All Available Courses

```
public void viewRegisteredCourses() {
    if(enrolledCourses.isEmpty()){
        JOptionPane.showMessageDialog(null, "You have not registered for any courses.");
        return;
    }
    StringBuilder msg = new StringBuilder("--- My Registered Courses ---\n");
    for(Course course : enrolledCourses){
        msg.append(course.toString()).append("\n");
    }
    JOptionPane.showMessageDialog(null, msg.toString());
}

Windsurf: Refactor | Explain | Generate Javadoc | X
public void viewAvailableCourses() {
    if(courses.isEmpty()){
        JOptionPane.showMessageDialog(null, "No courses available.");
        return;
    }
    StringBuilder msg = new StringBuilder("--- Available Courses ---\n");
    for(Course course : courses){
        msg.append(course.toString()).append("\n");
    }
    JOptionPane.showMessageDialog(null, msg.toString());
}
```

Figure 2.5: Student Course Viewing Interface

The methods `viewRegisteredCourses()` and `viewAvailableCourses()` are responsible for displaying course information to the student. The `viewRegisteredCourses()` method checks if the `enrolledCourses` list is empty, which indicates that the student has not enrolled in any courses yet. If the list is empty, it shows a message dialog informing the user that no courses have been registered. Otherwise, it displays the list of registered Courses. Similarly for the `viewAvailableCourses`, instead it shows all available Courses inside the system.

ADMIN

The Admin class accesses the list of all students, while a shared course list is maintained within the User base class to synchronize updates such as seat counts and enrollment changes across all users as stated below:

- Adding New Courses

```
public void addNewCourse() {
    String courseName = JOptionPane.showInputDialog(null, "Enter Course Name:");
    if (courseName == null) return;
    String courseCode = JOptionPane.showInputDialog(null, "Enter Course Code:");
    if (courseCode == null) return;
    String section = JOptionPane.showInputDialog(null, "Enter Section:");
    if (section == null) return;
    String creditHours = JOptionPane.showInputDialog(null, "Enter Credit Hours:");
    if (creditHours == null) return;
    String maxCapStr = JOptionPane.showInputDialog(null, "Enter Maximum Capacity:");
    if (maxCapStr == null) return;
    int maxCapacity;
    try {
        maxCapacity = Integer.parseInt(maxCapStr);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(null, "Invalid number for capacity", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Check duplicates
    for (Course course : courses) {
        if (course.getCourseCode().equalsIgnoreCase(courseCode)) {
            JOptionPane.showMessageDialog(null, "A course with this code already exists!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
    }

    Course newCourse = new Course(courseName, courseCode, section, creditHours, maxCapacity);
    courses.add(newCourse);
    JOptionPane.showMessageDialog(null, "Course added successfully!");
}
```

Figure 2.6: Admin Add New Course Interface

The method `addNewCourse()` enables administrators to input course information such as name, code, section, credit hours, and maximum capacity through a series of input dialogs. It includes input validation to ensure the capacity is a valid number and checks for duplicate course codes before creating and adding a new `Course` object to the shared list.

- Delete Available Courses

```
public void deleteCurrentCourse() {
    String courseCode = JOptionPane.showInputDialog(null, "Enter Course Code to Delete:");
    if (courseCode == null) return;
    Course toRemove = null;
    for (Course course : courses) {
        if (course.getCourseCode().equalsIgnoreCase(courseCode)) {
            toRemove = course;
            break;
        }
    }
    if (toRemove != null) {
        courses.remove(toRemove);
        for (Student student : students) {
            student.removeEnrolledStudents(toRemove);
        }
        JOptionPane.showMessageDialog(null, toRemove.getCourseName() + " deleted successfully!");
    } else {
        JOptionPane.showMessageDialog(null, "Course not found!", "Error", JOptionPane.ERROR_MESSAGE);
    }
}
```

Figure 2.7: Admin Delete Course Interface

The method `deleteCurrentCourse()` prompts the admin for a course code and, if the course exists, removes it from the system and from all enrolled student schedules to maintain consistency. If the course is not found, it shows an error message.

- View All Available Courses - Similar implementation to the `viewAllCourses` inside the `Student` class

- View Course Details

```
Windsurf: Refactor | Explain | Generate Javadoc | X
public void viewCourseDetails() {
    String courseCode = JOptionPane.showInputDialog(null, "Enter Course Code to View Details:");
    if (courseCode == null) return;
    for (Course course : courses) {
        if (course.getCourseCode().equalsIgnoreCase(courseCode)) {
            course.listDetails();
            return;
        }
    }
    JOptionPane.showMessageDialog(null, "Course not found!", "Error", JOptionPane.ERROR_MESSAGE);
}
```

Figure 2.8: Admin View Course Details Interface

The method above allows the admin to enter a specific course code and view detailed information for that course, including the list of enrolled student IDs, by calling the `listDetails()` method from the `Course` class.

- Viewing All Students

```
public void viewAllStudents() {
    if (students.isEmpty()) {
        JOptionPane.showMessageDialog(null, "No students found.");
        return;
    }
    StringBuilder msg = new StringBuilder("--- List of Students ---\n");
    for (int i = 0; i < students.size(); i++) {
        msg.append(i + 1).append(". ").append(students.get(i).getUserId()).append("\n");
    }
    JOptionPane.showMessageDialog(null, msg.toString());
}
```

Figure 2.9: Admin View All Students Interface

The method shows a list of all registered students by displaying their user IDs. It handles the case where no students are registered by showing an appropriate message.

COURSE

In the Course class, several methods manage course attributes and student enrollment. The constructor initializes all course details, including course name, code, section, credit hours, maximum capacity, and also prepares an empty list to track enrolled student IDs while setting the current capacity to zero. The details are explained below:

- Adding and Reducing Courses Capacities

```
public void minusCurrentCapacity() {  
    if (currentCapacity > 0) {  
        currentCapacity--;  
    }  
}  
  
Windsurf: Refactor | Explain | Generate Javadoc | ✕  
public void addEnrolledStudent(String studentId) {  
    if (!enrolledStudents.contains(studentId)) {  
        enrolledStudents.add(studentId);  
    }  
}
```

Figure 2.10: Course Capacity Management Methods

The addCurrentCapacity() method increments the current enrollment count when a new student registers, while the minusCurrentCapacity() method decrements this count when a student drops the course, ensuring it never goes below zero.

- Adding Enrolled Students

```
public void addEnrolledStudent(String studentId) {  
    if (!enrolledStudents.contains(studentId)) {  
        enrolledStudents.add(studentId);  
    }  
}
```

Figure 2.11: Add Enrolled Student Method

The method above appends a student's ID to the enrollment list if it is not already present, helping prevent duplicate entries.

- Viewing Courses enrolling details

```
public void listDetails() {  
    StringBuilder details = new StringBuilder();  
    details.append("Course Name: ").append(courseName).append("\n");  
    details.append("Course Code: ").append(courseCode).append("\n");  
    details.append("Section: ").append(section).append("\n");  
    details.append("Credit Hours: ").append(creditHours).append("\n");  
    details.append("Max Capacity: ").append(maxCapacity).append("\n");  
    details.append("Current Capacity: ").append(currentCapacity).append("\n\n");  
    if (enrolledStudents.isEmpty()) {  
        details.append("No students enrolled in this course.");  
    } else {  
        for (int i = 0; i < enrolledStudents.size(); i++) {  
            details.append(i + 1).append(". ").append(enrolledStudents.get(i)).append("\n");  
        }  
    }  
    JOptionPane.showMessageDialog(null, details.toString(), "Course Details", JOptionPane.INFORMATION_MESSAGE);  
}
```

Figure 2.12: Course Details Display Method

The listDetails() method compiles all course information and enrolled student IDs into a formatted string and displays it through a dialog, serving as a comprehensive view for administrators or students.

3. CLASS DIAGRAM

The class diagram represents the object-oriented design of a course registration system, highlighting the relationships between core components such as users, students, administrators, courses, and exception handling mechanisms. The system follows key object-oriented programming principles, including inheritance, aggregation, polymorphism, and exception handling.

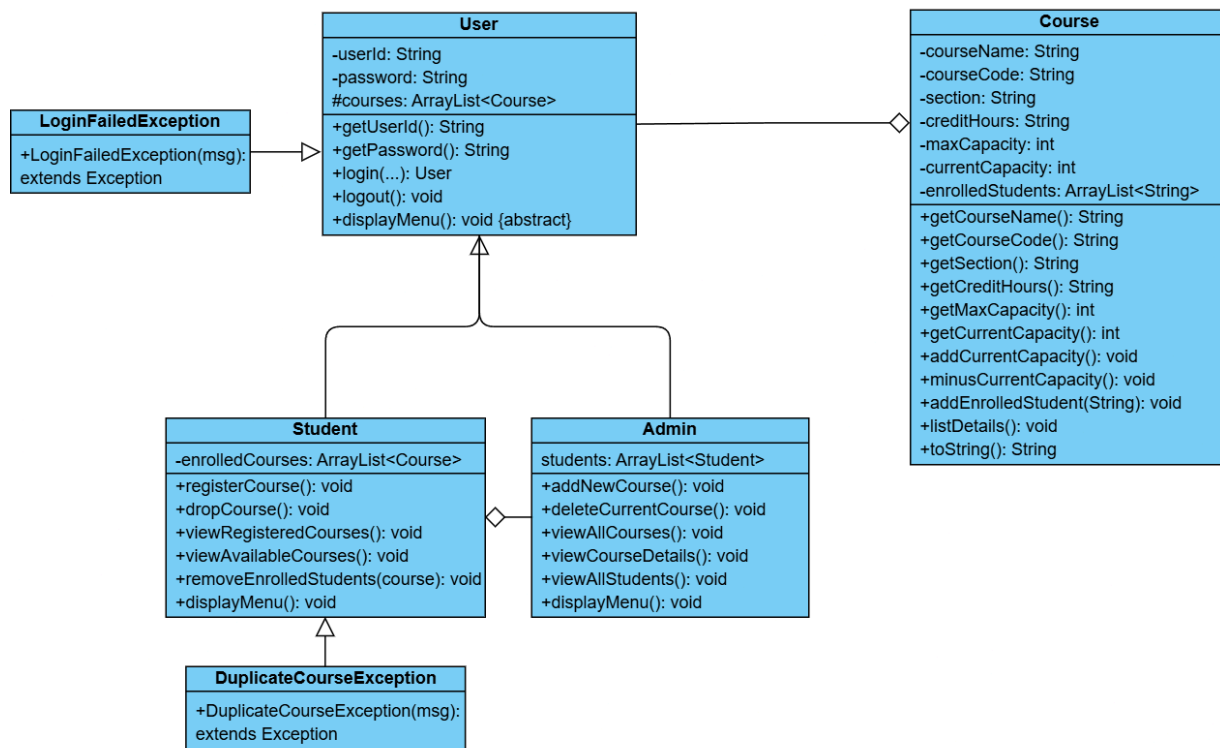


Figure 3.0: Class Diagram of the System

This class diagram outlines the core structure of the course registration system, built using key object-oriented principles like **inheritance**, **aggregation**, **polymorphism**, and **exception handling**.

At the top, the abstract class `User` acts as the base for both `Student` and `Admin`. Shared features like login, logout, and user credentials are defined here, reducing code repetition. This shows how **inheritance** allows us to reuse logic across different user roles.

The **Student** class has its own list of enrolled **Course** objects, while the **Admin** class manages a list of **Student** objects. Both classes also access a shared course list defined in **User**. These relationships reflect **aggregation** — a “has-a” connection that mirrors how students enroll in courses and admins manage them.

A good example of **polymorphism** can be seen in the **displayMenu()** method. Although both **Student** and **Admin** implement it differently, the system can call the same method regardless of user type. This makes the interface flexible and role-aware without extra complexity.

Finally, **exception handling** is integrated with two custom exceptions: **LoginFailedException** and **DuplicateCourseException**. These ensure the system responds gracefully to invalid logins or duplicate course registrations, keeping the experience smooth and error-aware.

In short, the class diagram reflects a system that’s organized, reusable, and easy to manage — with each class playing its role in a smart, interconnected way.

4. OBJECT-ORIENTED PROGRAMMING CONCEPTS DEMONSTRATED

INHERITANCE

inheritance is prominently featured, with both the **Student** and **Admin** classes extending the abstract **User** class. This approach promotes code reuse, as common logic like authentication and menu prompts is handled within the base class, while each subclass defines its specific actions.

AGGREGATION

Aggregation is another key concept demonstrated in the system. Each **Student** object maintains a personal list of the courses they are enrolled in, forming a "has-a" relationship. The **Admin** class accesses the list of all students, while a shared course list is maintained within the **User** base class to synchronize updates such as seat counts and enrollment changes across all users.

POLYMORPHISM

Polymorphism is implemented through the abstract method `displayMenu()`, which is declared in the `User` class and overridden in both the `Student` and `Admin` classes. This allows the system to display different menu options dynamically depending on whether the current user is a student or an administrator.

EXCEPTION HANDLING

exception handling is also incorporated to ensure robust and reliable user interaction. For instance, failed login attempts raise a `LoginFailedException`, providing feedback when incorrect credentials are entered. Additionally, a `DuplicateCourseException` is triggered if a student attempts to enroll in the same course more than once. Input validation is also applied to protect against invalid data types, particularly when administrators input course details like capacity or credit hours.

5. CONCLUSION

The Student Registration System exemplifies a well-structured Java application built using object-oriented principles. Its clear separation of roles between students and administrators, along with a shared data model for course management, ensures that the system remains consistent, scalable, and easy to maintain. By leveraging inheritance, aggregation, polymorphism, and exception handling, the application demonstrates sound design practices suitable for academic environments or as a prototype for larger educational systems. The use of `JOptionPane` for all user interactions further ensures simplicity and accessibility for both types of users without requiring a complex graphical user interface.

APPENDIX

<https://github.com/AdamRazal1/OOP--Mini-Project> - Project Github Repo

<https://youtu.be/Rcb3hk7L5cs> - Video Presentation

